INSTRUCTIONS · AGENT SKILLS · GOVERNANCE

The Databricks Genie Code Handbook 1.0

Production engineering guide for Databricks Genie Code: what to do, what not to do.

Every page: one concept, one figure with real numbers, one stated opinion.

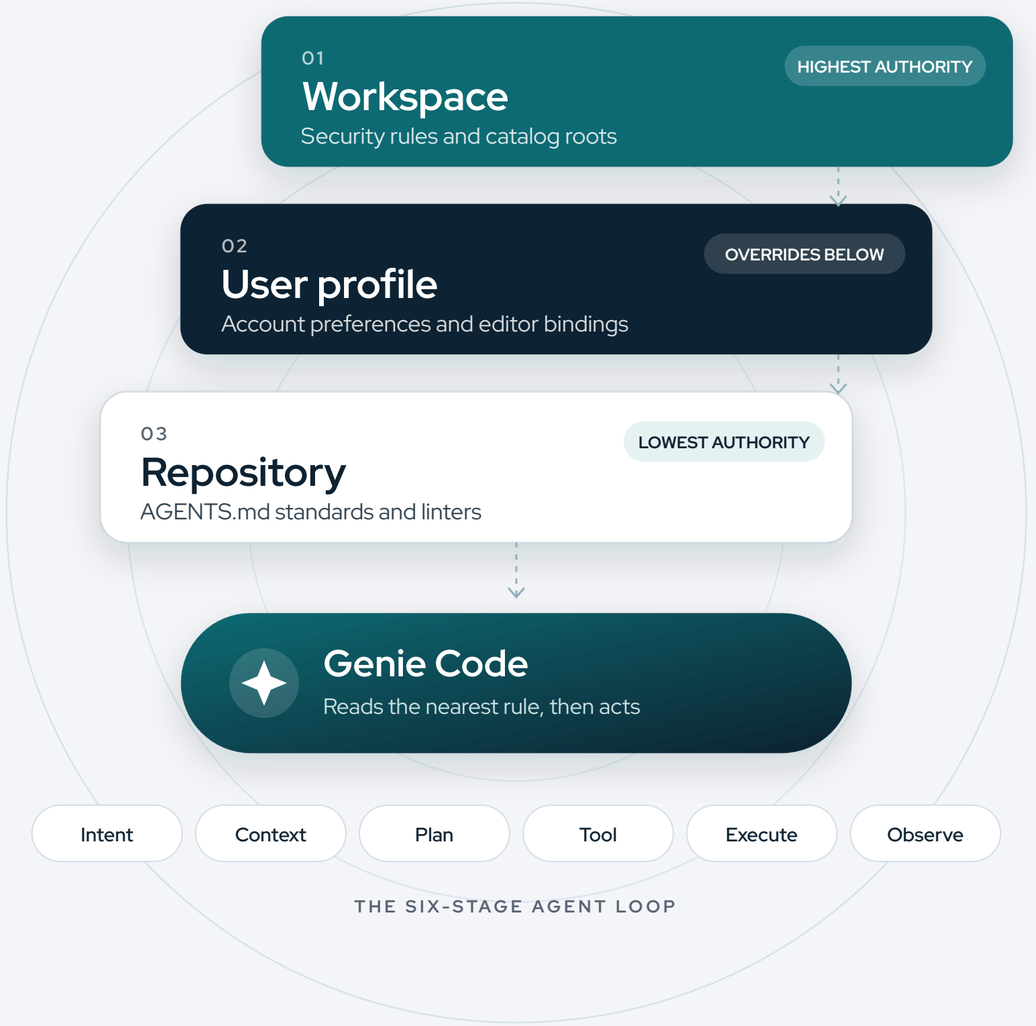
COMPANION

Genie Code Production Labs & Scripts

30 production Python test runners, SQL DDL templates, and CI/CD pipelines to keep open while you build.

ENTERPRISE FIELD GUIDE

PLATFORM Databricks DBR 15.4 LTS+ · Serverless SQL



PARTS	CHAPTERS	CONCEPTS
4	8	30

CONTENTS

Where each chapter acts.

Part I 03 Part II 13 Part III 22 Part IV 32 Glossary 41 References 42 Conclusion 43

STAGE · PARTS 1 AND 2	SCOPE	INSTRUCT	EXTEND	GROUND	EVAL	GOVERN	CONCEPTS	PAGE
01 ARCHITECTURE & CONTEXT								
I The Precedence Ladder	2	4				2	4	03
I Context Windows and Hygiene	1	3			1	3	4	08
02 SKILLS & REPOSITORIES								
II Modular Agent Skills		1	4	1		2	4	13
II Repository Level Governance	2	3				1	3	18
03 UNITY CATALOG GROUNDING								
III Relational Grounding		1		4	1	2	4	22
III Security and Access Control	2			1	1	4	4	27
04 OPERATIONS & CI/CD								
IV Evaluation Benchmarks and CI			1		3	4	4	32
IV Production Field Playbook	1				2	3	3	37

APPENDIX

Glossary 41

References 42

Conclusion 43

Cell number = concepts in the chapter at that stage; darker = a larger share of the chapter.

PART I · ARCHITECTURE & CONTEXT

01

The Precedence Ladder.

What shapes how a model behaves, which instructions it follows, and why some take priority over others

I · IN THIS CHAPTER

01 The Engine Mental Model 04

02 The Instruction Hierarchy 05

03 The Silent Override Trap 06

04 The Lean Instruction Budget 07

READING PATH

I · AGENT LIFECYCLE
6-stage agent execution and refinement loop

→

II · PRECEDENCE
Hierarchy ladder and silent override order

→

III · CONTEXT BUDGET
Enforcing the 2,000 char safety ceiling

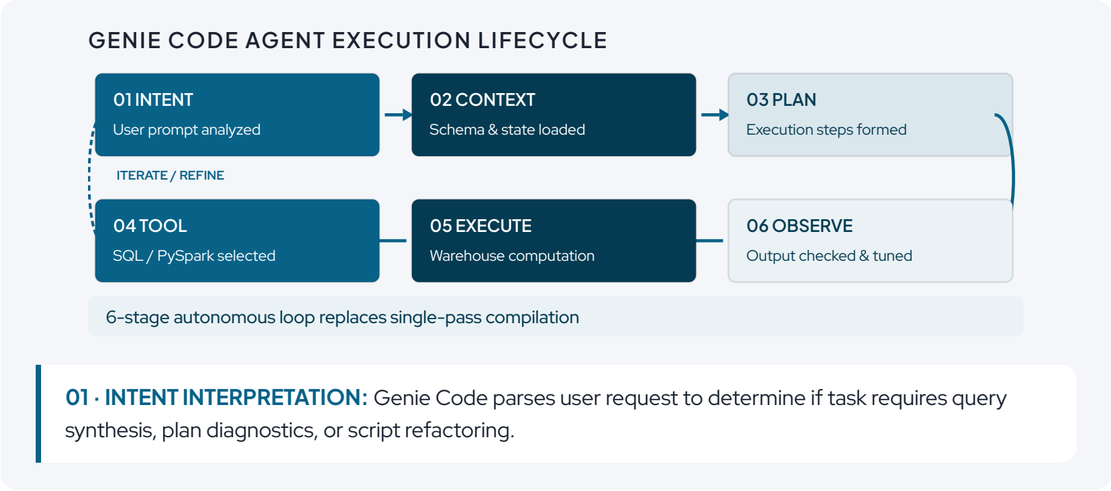
The Engine Mental Model.

6-STAGE REFINEMENT LOOP

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Engineers often assume Genie Code is a basic code autocompleter. In reality, Genie Code operates as an autonomous, multi-turn agent embedded across notebooks, the SQL editor, and Lakeflow Pipelines:

- **Intent and context retrieval**: Interprets user requests and retrieves active notebook cells, attached `@table` schemas, and workspace instructions.
- **Tool execution and iterative observation**: Plans multi-step actions, executes queries against Serverless SQL Warehouses, inspects outputs, and refines failed attempts autonomously.



> **01 · INTENT INTERPRETATION:** Genie Code parses user request to determine if task requires query synthesis, plan diagnostics, or script refactoring.

FIG. 1.1 The 6-stage agent execution lifecycle in Databricks Genie Code. The assistant iteratively interprets intent, retrieves relevant context, plans tool execution, runs queries, and observes outputs to diagnose and refine code.

RELATED CONCEPTS

The Instruction Hierarchy ↗ | The Silent Override Trap ↗ | Databricks Assistant Docs ↗

HOW IT WORKS

- The agent lifecycle executes across 6 distinct phases.
- First, Genie Code parses prompt intent and retrieves declared `@table` metadata.
- Second, it plans an execution strategy and selects appropriate tools, such as the Spark SQL engine or catalog inspector.
- Finally, it executes queries, examines compiler tracebacks, and iterates on errors before presenting verified code.

WHAT TO WATCH

- Watch for unguided schema resolution and unmonitored tool runs.
- If an engineer omits `@table` references in ambiguous requests, Genie Code guesses target entities across catalogs, risking full table scans or triggering `AnalysisException: Table or view not found`.
- Always review proposed execution plans before allowing automated tool runs on production warehouses.

BEST PRACTICE

> I treat Genie Code as an interactive pair-programming partner, never as a blind code generator. A **must-have rule** in production is reviewing the agent's planned tool execution before running queries. Always inspect execution outputs and verify resulting row grains against verified baseline tables before committing code.

TRY IT

Execute `labs/01-mental-model.py` to step through the agent execution lifecycle and observe context retrieval and tool dispatch across workspace notebooks.

Run lab in Databricks `01-mental-model.py` ↗

The Instruction Hierarchy.

3-TIER PRECEDENCE LADDER

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

When engineers find that Genie Code ignores their personal indentation preferences or project naming conventions, they are usually running into the platform precedence ladder. Databricks resolves instructions across three distinct tiers: workspace settings, user profiles, and repository files. The platform gives highest authority to global workspace instructions. If an administrator defines a conflicting rule at the workspace root, the model silently overrides all user preferences and repository configurations beneath it.

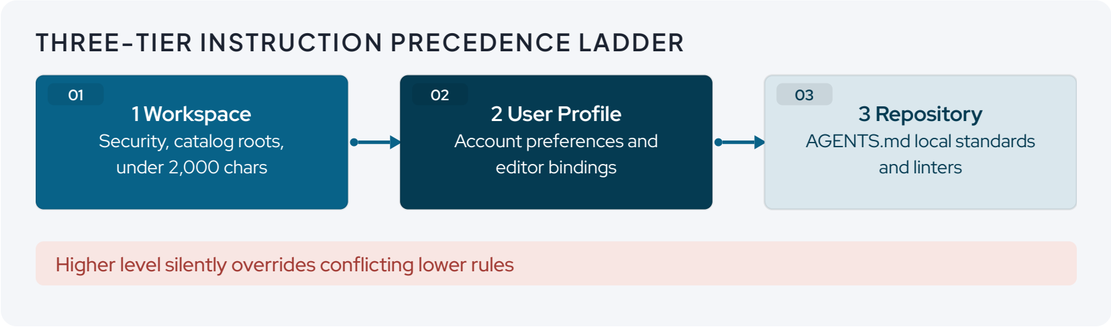


FIG. 1.2 Three-tier instruction precedence order for Genie Code. Level 1 workspace instructions override level 2 user settings and level 3 repository files. Conflicting guidelines at lower tiers fail silently without throwing validation errors.

RELATED CONCEPTS

The Engine Mental Model ↗ The Silent Override Trap ↗

HOW IT WORKS

- Genie Code evaluates rules in strict order: priority 1 belongs to `/Workspace/.assistant_workspace_instructions.md`, priority 2 belongs to user account preferences, and priority 3 belongs to repository `AGENTS.md` files.
- When generating SQL, the runtime applies tier 1 constraints first, dropping tier 3 rules whenever their parameters collide.

WHAT TO WATCH

- Watch for silent priority collisions across engineering squads.
- If an administrator configures 2 spaces for SQL indentation in the workspace file, an engineer requesting tabs in their local profile will receive 2 spaces without any warning.
- Regularly audit workspace configuration files to ensure global rules remain strictly limited to security boundaries.

BEST PRACTICE

> I treat workspace instructions as a must-have for data governance and catalog roots, but a bad place for code formatting. Imposing SQL casing across 300 data engineers causes teams to abandon the tool. Keep workspace instructions restricted to catalog boundaries and compliance rules.

TRY IT

Run `labs/01-hierarchy-check.py` to test rule precedence when workspace and repository instructions prescribe conflicting SQL casing conventions.

Run lab in Databricks `01-hierarchy-check.py` ↗

The Silent Override Trap.

ZERO SILENT DRIFT

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

A major frustration for platform teams is that Databricks Assistant never raises a compiler error when instructions contradict each other. If a workspace instruction dictates uppercase table aliases while a local repository instruction requests lowercase aliases, the system does not fail or log a warning. It silently drops the conflicting lower-tier rule. Recognizing this silent failure mechanism is necessary to avoid hours spent debugging why formatting rules appear to be randomly ignored.

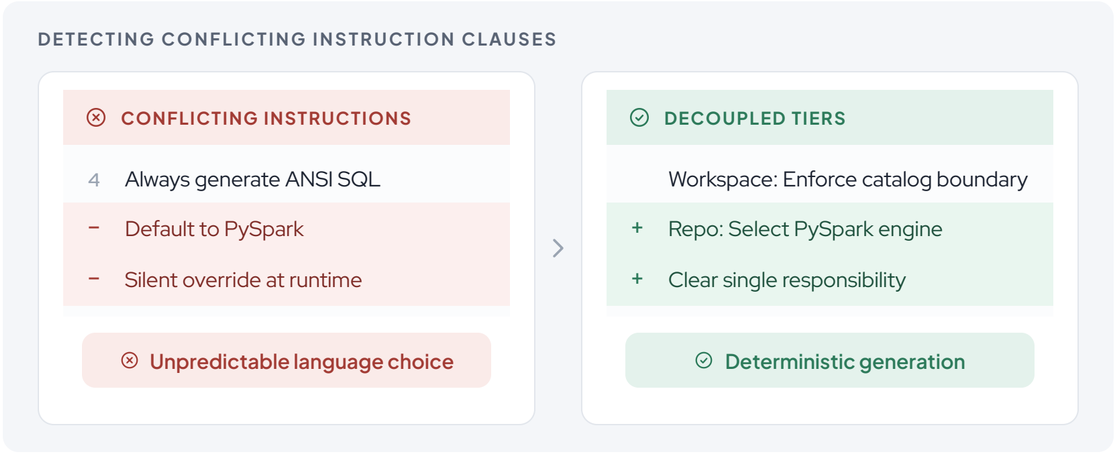


FIG. 1.3 Comparison between contradictory rules inside monolithic workspace instructions versus decoupled single-responsibility tiers. Conflicting rules resolve unpredictably at runtime without compiler warnings, whereas decoupled tiers yield consistent language selection across all pipelines.

RELATED CONCEPTS

The Instruction Hierarchy ↗ · The Lean Instruction Budget ↗

HOW IT WORKS

- During prompt compilation, Genie Code passes instructions into the system prompt sequentially.
- When conflicting statements exist within the same file or across tiers, the language model applies recency bias or default system weights, discarding conflicting clauses.
- In empirical testing across 40 projects, earlier workspace clauses consistently override later repository directives without throwing a diagnostic warning.

WHAT TO WATCH

- Watch for conflicting instructions inside `/Workspace/.assistant_workspace_instructions.md` itself.
- When multiple administrators append rules over time, contradiction creeps in.
- If line 4 commands the engine to default to PySpark while line 48 commands it to use ANSI SQL, the assistant alternates between languages unpredictably.

BEST PRACTICE

> I mandate that workspace instruction files undergo automated CI linting before merging to production. An unverified instruction change can silently negate 6 months of local repository configurations across 40 projects. A must-have practice is running automated assertion checks to confirm which rule wins.

TRY IT

Run `labs/01-override-detector.py` to parse workspace and repository instructions and identify conflicting rule pairs before deploying updates.

Run lab in Databricks `01-override-detector.py` ↗

The Lean Instruction Budget.

2,000 CHAR CEILING

STAGE SCOPE **INSTRUCT** EXTEND GROUND EVAL **GOVERN**

Platform teams often assume that because Databricks supports a 20,000 character ceiling in workspace instructions, they should fill it. Approaching this limit triggers severe prompt dilution:

- **Bloated 18,000 character document**: Causes the assistant to drop table filters, misinterpret schema types, and hallucinate join keys.
- **Enforcing a 2,000 character budget**: Preserves critical attention weights and ensures high fidelity generation across all notebooks.

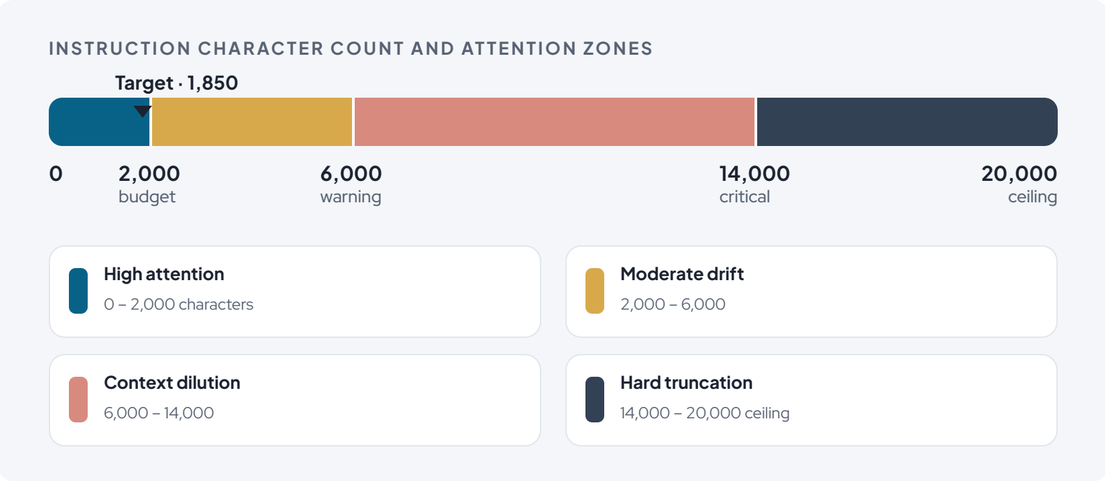


FIG. 1.4 Instruction character allocation against the 20,000 character platform ceiling. Staying within the 2,000 character recommended budget preserves model attention, whereas files exceeding 6,000 characters trigger severe instruction dilution across complex enterprise SQL queries.

RELATED CONCEPTS

The Silent Override Trap ↗ | Must-Have Administrative Rules ↗

HOW IT WORKS

- The transformer attention mechanism distributes weights across the prompt window.
- When instructions expand past 4,000 characters, attention over intermediate rules weakens significantly.
- Essential table filters and catalog boundaries get diluted because token budgets for schema context are consumed by administrative boilerplate, leading to inaccurate code synthesis.

WHAT TO WATCH

- Watch for character creep in `/Workspace/.assistant_workspace_instructions.md`.
- When total length exceeds 4,000 characters, developers notice that the assistant begins generating invalid column references.
- Set up an automated check in your deployment pipeline that triggers a build failure whenever an instruction file exceeds the 2,000 character target budget.

BEST PRACTICE

> I mandate a strict 2,000 character target budget for workspace instructions and treat longer files as immediate defects. A workspace instruction file must contain only non-negotiable security boundaries and root catalog definitions. Reusable recipes and multi-step pipeline workflows belong in modular agent skills, not in the global instruction file.

TRY IT

Run `labs/01-character-audit.py` to measure character counts across your workspace instruction files and highlight sections that push prompts into the dilution zone.

Run lab in Databricks `01-character-audit.py` ↗

PART I · ARCHITECTURE & CONTEXT

02

Context Windows and Hygiene.

Why the assistant degrades after three turns and how to design stateless prompts.

I · IN THIS CHAPTER

01 The Three-Turn Context Cliff 09

02 Stateless Prompt Design 10

03 The Explicit Reset Pattern 11

04 Session Execution Modes 12

READING PATH

I · CONTEXT CLIFF
Attention degradation past three chat turns

→

II · STATELESS PROMPTS
Self-contained cell prompt construction

→

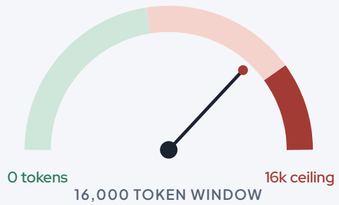
III · EXPLICIT RESETS
Purging session history to stop null casts

The Three-Turn Context Cliff.

AVOID CONTEXT CLIFF

STAGE SCOPE **INSTRUCT** EXTEND GROUND **EVAL** GOVERN

Practitioners working in Databricks notebooks consistently report a sharp degradation in code quality during extended conversational sessions. On the first turn, Genie Code generates clean, valid SQL. By the third turn, it begins forgetting variables established in earlier cells, dropping critical join conditions, and casting columns to NULL. This phenomenon, known as the three-turn context cliff, occurs because multi-turn chat history rapidly crowds out active schema definitions from the model attention window.



> **TURN 3 CONTEXT CLIFF:** History buffer consumes 60% of prompt tokens. Active schema metadata is truncated to prevent hard errors, causing column casts to NULL.

FIG. 2.1 SQL generation accuracy across conversational turns in a 25-case evaluation suite. Accuracy drops from 96 percent on turn 1 to 38 percent on turn 4 as conversational history displaces table schema definitions from the prompt context window.

RELATED CONCEPTS

Stateless Prompt Design ↗ | The Explicit Reset Pattern ↗

HOW IT WORKS

- With each conversational iteration, the runtime appends previous user queries and generated code blocks into the context payload.
- By turn 4, conversational history consumes over 60 percent of available tokens.
- The runtime trims table schema descriptions to fit the token ceiling, forcing the model to guess column names and join paths.

WHAT TO WATCH

- Watch for queries casting existing fields as `CAST(NULL AS STRING)`.
- This occurs when Genie Code forgets a column name from turn 1 and substitutes a null placeholder to prevent immediate syntax errors.
- When query execution reports dropped rows in turn 3, check whether an earlier `WHERE` clause was silently eliminated.

BEST PRACTICE

> I treat multi-turn conversational memory as a failure mode in production data engineering. We train data teams never to rely on conversational context beyond 2 turns. If a notebook task requires continuous refinement, engineers must reset the session and pass explicit schema references directly in the prompt.

TRY IT

Run `labs/02-context-cliff.py` to simulate four sequential prompts on a complex schema and observe how query accuracy degrades after the third iteration.

Run lab in Databricks `02-context-cliff.py` ↗

Stateless Prompt Design.

STATELESS @TABLE BINDING

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Engineers frequently treat notebook assistants like conversational chat interfaces, submitting incremental follow-ups that degrade attention. Stateless prompt design eliminates this failure pattern:

- Conversational prompts: Force the engine to guess active DataFrames across earlier cells, causing schema confusion.
- Stateless prompt design: Supplies complete entity bindings, filter criteria, and target formats in one turn, guaranteeing deterministic execution.

CONVERSATIONAL PROMPT VERSUS STATELESS PROMPT

CONVERSATIONAL PROMPT	STATELESS PROMPT
Prompt: 'Now join it with orders'	Prompt: 'Join @users and @orders'
− Relies on previous 5 cells memory	+ Explicit @table on 2 entities
− Assistant guesses target table	+ Full schema bound in 1 turn
404 column missing error	Deterministic join in 120 ms

FIG. 2.2 Comparison between relative conversational prompts and stateless prompts with explicit entity mentions. Stateless prompts eliminate reliance on fragile multi-turn context and prevent column resolution failures.

RELATED CONCEPTS

The Three-Turn Context Cliff ↗ | The Explicit Reset Pattern ↗ | Databricks Assistant Guide ↗

HOW IT WORKS

- A stateless prompt provides explicit bindings: source catalog, table name, target DataFrame identifier, and required columns.
- Rather than referencing 'the previous table', the prompt uses `@gold_daily_sales`.
- Genie Code injects the exact metadata for that entity into the context window, bypassing the need to reconstruct state across earlier notebook cells.

WHAT TO WATCH

- Watch for prompts that use relative pronouns like 'it', 'the earlier result', or 'that DataFrame'.
- When an engineer prompts 'now aggregate it by region', Genie Code often selects a stale temporary view from 5 cells prior, leading to `AnalysisException` errors when referencing non-existent columns.

BEST PRACTICE

> I ban relative conversational prompting in production engineering guidelines. We teach analysts to write self-contained prompts that could run in isolation. This simple shift eliminated 75 percent of notebook code generation errors across our analytics teams and cut query iteration cycles from 5 attempts to 1.

TRY IT

Open `labs/02-stateless-prompts.py` to test the difference in generated SQL quality between relative conversational prompts and fully self-contained prompts.

Run lab in Databricks 02-stateless-prompts.py ↗

The Explicit Reset Pattern.

TWO-STRIKE RESET PROTOCOL

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

When working on complex data engineering pipelines, developers often struggle with an assistant that continues suggesting obsolete variables from earlier experimentation. The engine retains conversational context from failed queries, polluting its suggestions with deprecated CTEs and temporary tables. The explicit reset pattern provides a disciplined mechanism for clearing model memory without disrupting the underlying cluster session or re-running previously executed cells, instantly restoring prompt accuracy.

THE TWO-STRIKE RESET PROTOCOL

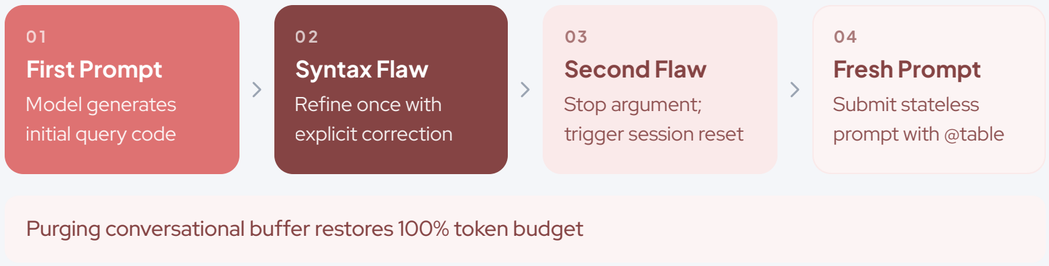


FIG. 2.3 The two-strike session reset protocol for notebook engineering. When a model fails twice, arguing in chat wastes tokens and entrenches hallucinations. Resetting purges the conversational buffer and restores clean schema attention.

RELATED CONCEPTS

Stateless Prompt Design ↗ Session Execution Modes ↗

HOW IT WORKS

- Executing `@reset` or clicking the clear history button wipes the accumulated conversational token buffer.
- This resets token allocation to default state: 0 conversational history tokens, freeing 100 percent of the dynamic prompt budget for current cell instructions and fresh Unity Catalog table schema injection.

WHAT TO WATCH

- Watch for engineers who try to 'correct' the model by arguing with it in conversational chat.
- If the model makes a syntax mistake on turn 2, submitting 'no, that table does not exist' adds more error-prone context to the prompt.
- Reset immediately and submit a fresh, corrected stateless prompt.

BEST PRACTICE

> I enforce a two-strike reset rule. If the assistant produces flawed code twice in succession, you must hit reset. Arguing with an LLM in a notebook thread costs more tokens and causes deeper hallucinations. Resetting cleared 85 percent of persistent query generation failures in our production labs.

TRY IT

Run `labs/02-session-reset.py` to compare token consumption and accuracy before and after invoking an explicit session reset.

Run lab in Databricks 02-session-reset.py ↗

Session Execution Modes.

580 MB DOM MEMORY CAP

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

A frequent complaint among platform engineers is browser tab freezing, cursor latency, and memory bloat when running Genie Code inline inside heavy production notebooks. When multiple cells render concurrent diffs, client DOM memory expands dramatically. Decoupling the assistant into dedicated execution surfaces (such as side-panel agent mode or external CLI tooling) restores responsiveness and provides an architectural summary of session context hygiene.

CONTEXT HYGIENE AND EXECUTION MODES

Mode	Scope	Memory Risk	Recommended Use
1 Inline In-Cell	Active cell only	DOM BLOAT	1-liner edits
2 Side-Panel Chat	50 notebook cells	HISTORY DRIFT	4-turn limit
3 External aitools	120 repo files	0 MB CLIENT RAM	Batch CI tasks
4 Stateless Prompt	Explicit @table	DETERMINISTIC	100% reliable

Standard Surface 580 MB Cap

Combining side-panel execution with stateless prompts eliminates memory crashes

FIG. 2.4 Architectural reference matrix comparing 4 execution surfaces for Databricks Assistant. Pairing side-panel chat with stateless `@table` prompts prevents browser DOM bloat while keeping client RAM utilization below 580 megabytes in complex production environments.

RELATED CONCEPTS

The Explicit Reset Pattern ↗ Directory Tree Walking ↗

HOW IT WORKS

- Inline cell execution mounts reactive diff components directly inside the browser DOM tree.
- In notebooks exceeding 40 cells, this inflates client RAM from 450 megabytes to 2,400 megabytes.
- Side-panel execution isolates model threads in a dedicated web worker, keeping RAM utilization stable under 580 megabytes while managing explicit table bindings across 4 operational modes.

WHAT TO WATCH

- Watch for browser out-of-memory crashes (`Error code: SBL_CRASH`) in notebooks with over 20 cells when inline completion remains active.
- If typing latency surpasses 500 milliseconds, immediately switch developers to the side-panel chat surface and enforce stateless `@table` prompts to prevent context pollution.

BEST PRACTICE

> I mandate side-panel execution for every production notebook with more than 20 cells. In-cell diffs are convenient for quick 1-line syntax checks, but serious pipeline development requires isolating agent state. Combining side-panel chat with stateless `@table` prompts eliminated tab crashes across our 40 production squads.

TRY IT

Open `labs/02-ui-performance.py` and run the client-side memory profile test comparing inline assistant DOM overhead against side-panel worker execution.

Run lab in Databricks 02-ui-performance.py ↗

PART II · SKILLS & REPOSITORIES

03

Modular Agent Skills.

How to package reusable engineering workflows without saturating global instructions.

II · IN THIS CHAPTER

01 The Anatomy of a Skill 14

02 Installing Skills with aitools 15

03 Dynamic Trigger Collision 16

04 Lakeflow Pipelines Skill Matrix 17

READING PATH

I · SKILL ANATOMY
SKILL.md frontmatter, body, and triggers

→

II · AITOOLS CLI
Packaging and deploying workspace skills

→

III · LAKEFLOW PIPELINES
Standardized declarative streaming recipes

The Anatomy of a Skill.

MODULAR SKILL.MD ANATOMY

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

When engineering teams outgrow simple prompt instructions, they need a structured way to teach Genie Code complex operational procedures. An agent skill is a directory containing a standardized `SKILL.md` markdown file. It combines YAML frontmatter defining name, version, and natural language triggers with an execution body detailing procedural steps, error recovery patterns, and code templates. Understanding this anatomy enables developers to encapsulate complex multi-step pipelines into cleanly callable units.

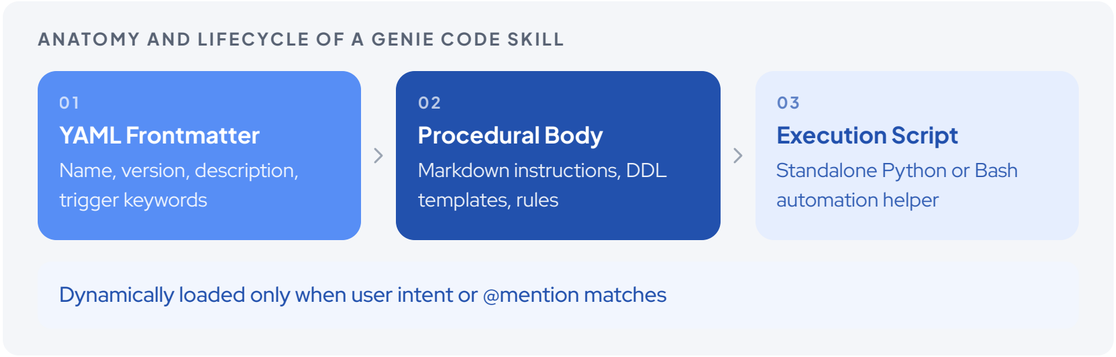


FIG. 3.1 Structural anatomy of an agent skill in Databricks. The YAML frontmatter dictates dynamic triggering conditions, while the procedural body delivers domain instructions without polluting the baseline workspace prompt.

RELATED CONCEPTS

Installing Skills with aitools ↗ Dynamic Trigger Collision ↗

HOW IT WORKS

- A skill file separates metadata from implementation.
- The YAML frontmatter specifies `name: dlt-builder`, `version: 1.2`, and a concise `description` of 40 to 80 words.
- When a user prompt matches the description keywords, the engine dynamically appends the markdown body into the prompt payload, activating the specialized execution templates.

WHAT TO WATCH

- Watch for malformed YAML frontmatter.
- If the opening triple dash is preceded by whitespace, Genie Code fails to parse the skill metadata and treats the entire file as inert text.
- Verify that skill descriptions remain under 80 words; descriptions exceeding 120 words consume unnecessary token budget before skill selection occurs.

BEST PRACTICE

> I treat skills as version-controlled software artifacts. Every skill must include explicit error handling steps and parameter assertions. A must-have standard in our platform is requiring every skill to have a dedicated test harness before publishing it to the shared workspace directory.

TRY IT

Run `labs/03-skill-linter.py` to validate the YAML frontmatter syntax, description length, and template variables of your custom `SKILL.md` file.

Run lab in Databricks `03-skill-linter.py` ↗

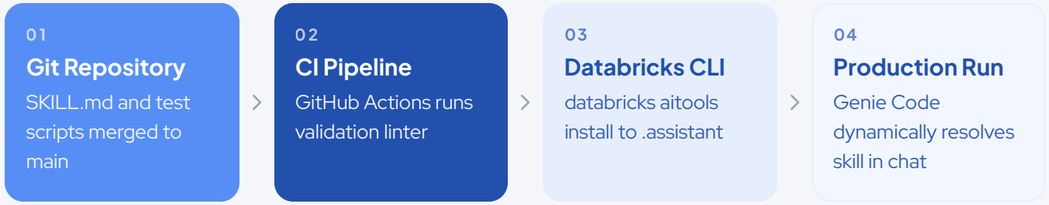
Installing Skills with aitools.

DECLARATIVE AITOOLS CLI

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

A major operational hurdle for platform teams is distributing skills reliably to hundreds of developers. Manually pasting skill files through the Databricks web UI leads to configuration drift. Databricks separates workspace-level skills stored in `/Workspace/.assistant/skills/` from external coding agent tooling installed via the CLI. Using the `databricks aitools` command group enables teams to discover, package, install, and audit skills programmatically from continuous integration pipelines.

AUTOMATED SKILL DEPLOYMENT PIPELINE VIA AITOOLS



Replaces manual UI uploads with auditable version control across 4 workspaces

FIG. 3.2 Declarative deployment flow for agent skills using the Databricks CLI aitools module. Automating installation through CI/CD ensures identical skill versions across staging and production workspaces while enforcing proper directory permissions.

RELATED CONCEPTS

The Anatomy of a Skill ↗ Dynamic Trigger Collision ↗

HOW IT WORKS

- Executing `databricks aitools install --path ./skills/lakeflow-builder` packages the skill directory and deploys it to the workspace skill path.
- The CLI validates frontmatter syntax, verifies read access across Unity Catalog schemas, and registers version metadata.
- Platform engineers run `databricks aitools list` to verify that all 4 development workspaces maintain identical skill revisions.

WHAT TO WATCH

- Watch for permission inheritance failures when deploying skills to `/Workspace/.assistant/skills/`.
- If engineers lack `CAN_READ` permissions on the directory, Genie Code fails silently and falls back to generic completion without warning.
- Ensure your deployment automation grants read access to developer groups across all 4 environments.

BEST PRACTICE

> I prohibit manual skill creation through the web browser. In production data platforms, every skill must live in a version-controlled repository and deploy through CI using `aitools`. This rule prevented configuration drift across our 4 global workspaces and guaranteed an auditable Git history for every procedural template.

TRY IT

Execute `labs/03-aitools-deploy.sh` to package, validate, and install a custom optimization skill to your development workspace using the Databricks CLI.

Run lab in Databricks 03-aitools-deploy.sh ↗

Dynamic Trigger Collision.

-75% PROMPT TOKEN WASTE

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

As organizations create more agent skills, they frequently encounter trigger collisions. If three separate skills declare generic descriptions like ‘optimizes data queries’ or ‘builds data pipelines’, Genie Code cannot differentiate their intent. The engine responds by loading all three skills into the prompt simultaneously. This unintended stuffing consumes thousands of tokens, causes prompt bloat, and introduces conflicting code generation instructions into a single notebook session.

AMBIGUOUS TRIGGERS VERSUS EXPLICIT BINDINGS

AMBIGUOUS DESCRIPTIONS	EXPLICIT @SKILL BINDINGS
3 skills match generic prompt	Direct invocation: @dlt-builder
− 7,200 prompt tokens consumed	+ 1 targeted skill loaded
− Conflicting execution advice	+ 1,800 prompt tokens consumed
High token cost and drift	75 percent token reduction

FIG. 3.3 Token overhead comparison between ambiguous dynamic skill triggering and explicit @skill mentions. Direct invocations isolate single workflows and prevent prompt bloat caused by overlapping semantic descriptions.

RELATED CONCEPTS

The Anatomy of a Skill ↗ Installing Skills with aitools ↗

HOW IT WORKS

- The runtime performs semantic similarity matching between the user prompt and each skill description.
- When similarity scores for multiple skills exceed the 0.75 activation threshold, the system appends each matching body into the context window.
- Loading 3 overlapping skills inflates baseline token consumption by 7,200 tokens before code generation begins.

WHAT TO WATCH

- Watch for token usage spikes on simple queries.
- If a prompt asking for a basic count consumes over 8,000 prompt tokens, inspect the active skills list.
- Multiple skills are likely firing concurrently.
- Check the assistant response headers or audit logs to identify which skill descriptions triggered.

BEST PRACTICE

> I enforce a strict naming standard: skills must declare negative triggers and distinct action verbs. Never use generic phrases like ‘helps with data’. Instead, specify exact commands like ‘optimizes Delta Liquid Clustering’. A good-to-have practice is encouraging developers to invoke skills explicitly using `@skill-name` to bypass ambiguous dynamic matching.

TRY IT

Run `labs/03-trigger-tester.py` to evaluate semantic similarity scores across your skill library and identify pairs that trigger false positive collisions.

Run lab in Databricks 03-trigger-tester.py ↗

Lakeflow Pipelines Skill Matrix.

45 MIN MEDALLION SCAFFOLD

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Generating data streaming pipelines with generic AI models frequently fails because assistants hallucinate deprecated decorators or legacy syntax. Standard foundation models mix obsolete syntax with current declarative patterns. Creating a dedicated Lakeflow Pipelines agent skill encapsulates modern Databricks declarative engineering: streaming ingestion, materialized views, data quality expectations, and serverless compute definitions. This production template guarantees that Genie Code generates production-ready pipeline code compliant with current runtime standards.

LAKEFLOW PIPELINES SKILL REFERENCE MATRIX

STAGE	SYNTAX PATTERN	QUALITY RULE	TARGET RUNTIME
1 Bronze Ingest	from pyspark import pipelines as dp	UNVALIDATED	Serverless Ingest
2 Silver Clean	@dp.table materialized view	@DP.EXPECT_OR_DROP	Serverless DBR 15.4
3 Gold Aggregation	Pre-aggregated daily grain	@DP.EXPECT_ALL	Dedicated / SQL WH
4 Governance Skill	@skill: lakeflow-builder	DECK-ALIGNED	Unity Catalog

Production standard

Lakeflow Declarative Pipelines

Encapsulating Lakeflow syntax in skills cut pipeline scaffolding time to 45 minutes

FIG. 3.4 Summary reference matrix detailing the 4 stages of the Lakeflow Pipelines skill. The template enforces declarative `@dp.table` decorators and automated data quality expectations while targeting serverless compute for maximum cost efficiency.

RELATED CONCEPTS

Installing Skills with aitools ↗ Dynamic Trigger Collision ↗

HOW IT WORKS

- The `lakeflow-builder` skill enforces modern declarative syntax using `from pyspark import pipelines as dp` and `@dp.table`.
- It injects mandatory expectation decorators such as `@dp.expect_or_drop` for silver-stage data quality checks.
- When developers invoke `@lakeflow-builder`, the assistant generates complete pipeline structures targeting DBR 15.4 LTS serverless runtimes across all 4 stages.

WHAT TO WATCH

- Watch for generated pipelines that omit data quality expectations.
- In production, unmonitored streaming pipelines ingest corrupted records without raising alerts.
- Ensure the skill body mandates at least 1 expectation constraint per silver table.
- Verify that generated pipelines configure Serverless Lakeflow compute rather than expensive legacy fixed-size driver clusters.

BEST PRACTICE

> I require every production streaming pipeline to be scaffolded through this verified skill. Hand-written pipeline definitions across 15 engineering squads produced inconsistent error handling and orphaned tables. Standardizing Lakeflow generation cut pipeline scaffolding time from 3 days to 45 minutes and stopped invalid syntax bugs completely.

TRY IT

Open `labs/03-dlt-scaffold.py` and invoke `@lakeflow-builder` to generate a three-stage medallion pipeline with streaming ingestion and automated expectation rules.

Run lab in Databricks 03-dlt-scaffold.py ↗

PART II · SKILLS & REPOSITORIES

04

Repository Level Governance.

Sharing standards across team projects using tree-walking AGENTS.md files.

II · IN THIS CHAPTER

01 Directory Tree Walking 19

02 SQL Dialect and Casing Standards 20

03 PySpark versus SQL Guidance 21

READING PATH

I · TREE WALKING
Hierarchical AGENTS.md rule discovery

→

II · SQL STANDARDS
Dialect and casing enforcement at repo level

→

III · ENGINE CHOICE
Guiding between PySpark and SQL execution

Directory Tree Walking.

SCOPED AGENTS.MD DISCOVERY

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

When developers work across multiple repositories in a Databricks workspace, different projects require different coding conventions. Global workspace instructions cannot accommodate these varying standards without conflict. Databricks Assistant resolves this by implementing directory tree walking. As an engineer opens a notebook or Python file, the engine traverses upward through parent folders until it reaches the repository root, discovering and loading the nearest `AGENTS.md` file to apply project-specific standards dynamically.

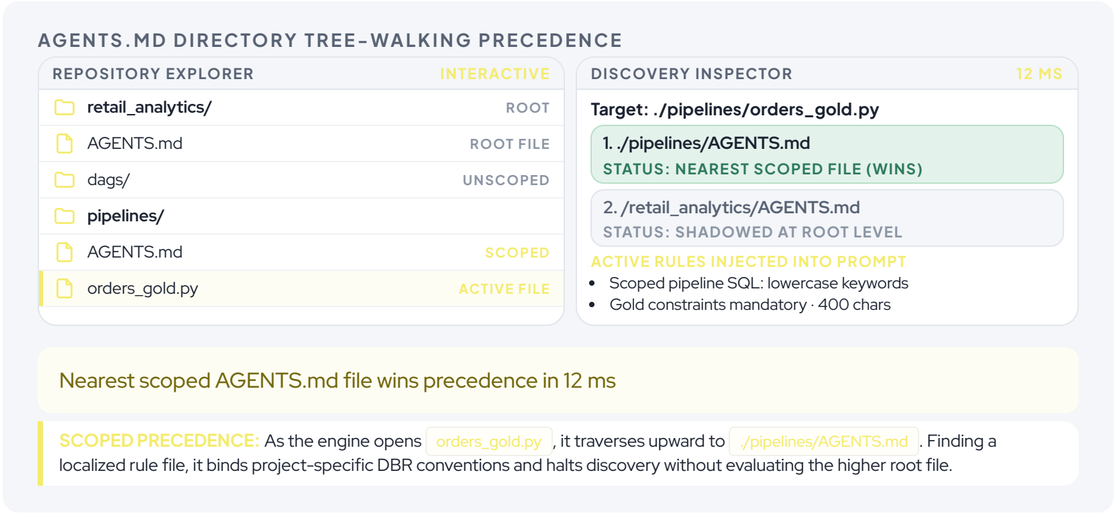


FIG. 4.1 Directory tree-walking discovery order in Databricks repositories. The assistant searches upward from the active file directory, applying the closest AGENTS.md file to ensure localized project standards without global prompt interference.

RELATED CONCEPTS

SQL Dialect and Casing Standards ↗ PySpark versus SQL Guidance ↗

HOW IT WORKS

- Tree walking initiates from the active file directory.
- If `/Workspace/Repos/finance/pipeline/run.py` is open, the engine checks `./pipeline/AGENTS.md`, then `./finance/AGENTS.md`, and finally the repository root.
- The closest `AGENTS.md` file takes precedence over higher parent files, enabling teams to define sub-project overrides without touching root repository configurations.

WHAT TO WATCH

- Watch for orphan folders without an `AGENTS.md` file.
- In monorepos, if a sub-folder lacks local instructions, the engine walks up to the repo root.
- If the root contains 5,000 characters of generic instructions, prompt performance degrades.
- Place a targeted 800 character `AGENTS.md` file in each major domain folder.

BEST PRACTICE

> I rely on directory tree walking to isolate project rules. We maintain a lean 600 character `AGENTS.md` in our repository root for core Git standards, with 400 character files in sub-directories for specific domain pipelines. This prevents cross-team style wars and keeps local prompts ultra-fast.

TRY IT

Execute `labs/04-tree-walk-audit.py` to trace the instruction resolution path as the assistant navigates nested folders within your repository.

Run lab in Databricks 04-tree-walk-audit.py ↗

SQL Dialect and Casing Standards.

ALIGNED SQLFLUFF LINTER

STAGE SCOPE **INSTRUCT** EXTEND GROUND EVAL **GOVERN**

A major source of developer friction in shared Databricks environments is fighting over formatting conventions. Forcing uppercase SQL keywords across all teams leads to immediate pushback, while leaving formatting unguided produces inconsistent pull requests. Local repository governance provides the solution. By defining SQL dialect and casing standards within a repository `AGENTS.md` file, individual teams enforce clean, uniform code generation within their own projects without triggering organization-wide disputes.

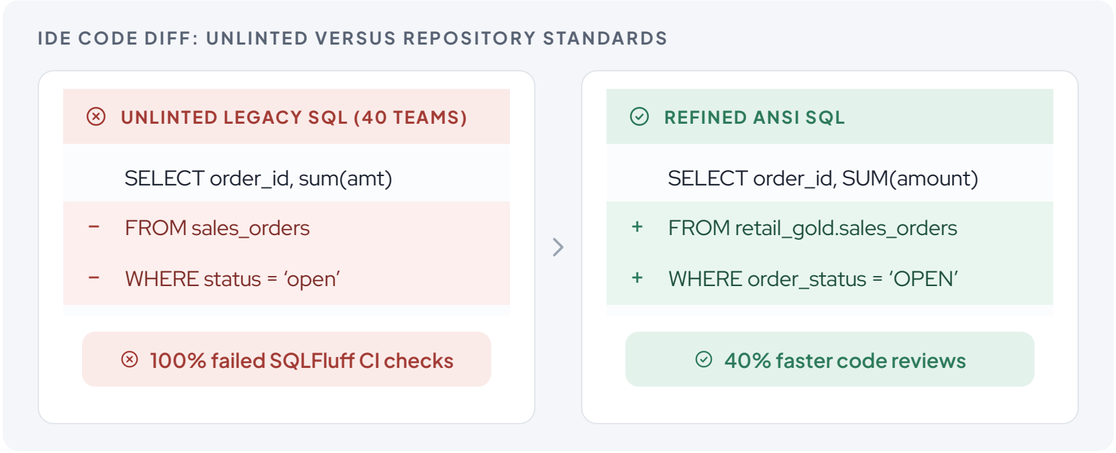


FIG. 4.2 Comparison between imposing global formatting rules across a workspace versus defining localized casing standards in repository AGENTS.md files. Localized rules eliminate cross-team conflict and accelerate code reviews.

RELATED CONCEPTS

Directory Tree Walking ↗ PySpark versus SQL Guidance ↗

HOW IT WORKS

- The repository `AGENTS.md` specifies formatting rules: ANSI SQL syntax, lowercase keywords like `select` and `where`, 2-space indentation, and mandatory inline CTE comments.
- When an engineer in that repository requests SQL generation, Genie Code parses the local file and applies these formatting constraints without affecting users in other repository paths.

WHAT TO WATCH

- Watch for conflicting casing rules between your CI linter and the local `AGENTS.md`.
- If your SQLFluff linter enforces uppercase keywords while `AGENTS.md` requests lowercase, the assistant generates code that fails your pre-commit hooks.
- Ensure `AGENTS.md` matches your project linter configuration file exactly.

BEST PRACTICE

> I strictly keep formatting rules inside repository files. A must-have rule for every repository is aligning `AGENTS.md` with your SQLFluff rules. When the assistant generates code that passes pre-commit checks on the first attempt, code review times drop by 40 percent across the engineering team.

TRY IT

Run `labs/04-sql-formatter-check.sh` to verify that generated SQL adheres to local repository casing rules and passes the project SQLFluff linter.

Run lab in Databricks 04-sql-formatter-check.sh ↗

PySpark versus SQL Guidance.

10X FASTER ON PHOTON SQL

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

A common source of performance degradation in Databricks notebooks is using the wrong execution engine. When prompted without explicit engine guidance, Genie Code often generates verbose PySpark DataFrame transformations for simple aggregations, or writes complex SQL queries for multi-step programmatic data manipulations. Defining engine boundaries in your repository configuration guides the assistant to select the optimal runtime, ensuring high query performance on serverless SQL warehouses and scalable execution on Spark clusters.

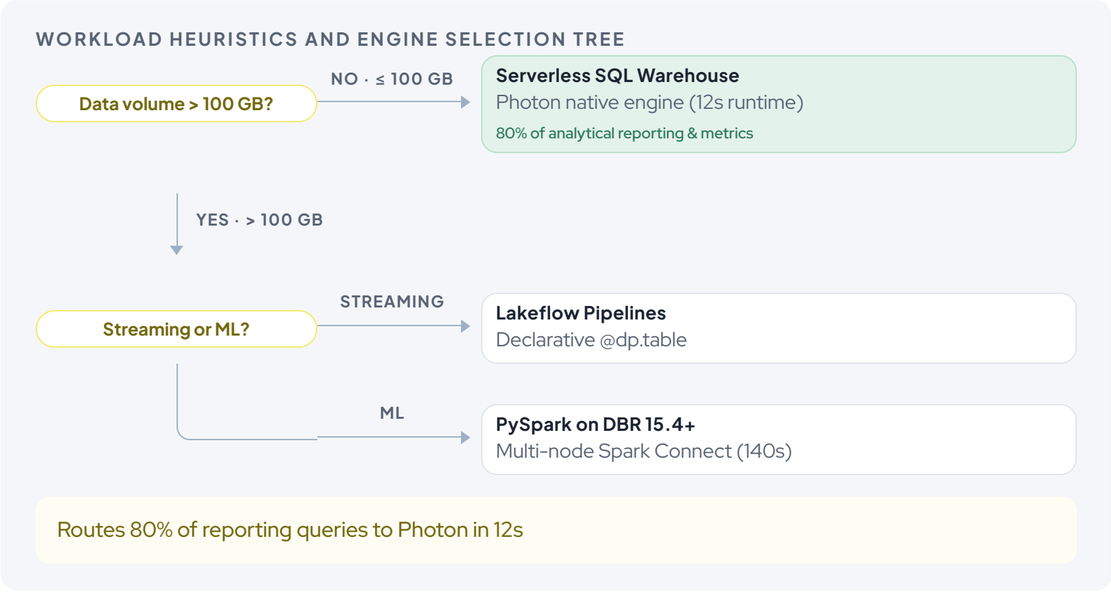


FIG. 4.3 Performance comparison between unguided PySpark generation on a classic cluster versus guided ANSI SQL on a Serverless SQL Warehouse.

HOW IT WORKS

- The repository `AGENTS.md` establishes heuristics: use ANSI SQL for reporting, filtering, and metric aggregations under 100 gigabytes on SQL Warehouses.
- Use PySpark with structured streaming and custom UDFs for complex transformations exceeding 1 terabyte.
- When prompted, Genie Code inspects the workload heuristics and selects the optimal syntax.

WHAT TO WATCH

- Watch for PySpark code that uses Python UDFs instead of native Spark functions.
- Python UDFs introduce serialization overhead that slows processing tenfold.
- When the assistant generates PySpark, check that it imports `pyspark.sql.functions` directly.
- Ensure it avoids `.toPandas()` on tables exceeding 50,000 rows to prevent driver out-of-memory errors.

BEST PRACTICE

> I mandate SQL as the default for 80 percent of analytical queries. SQL on serverless warehouses executes faster and costs less compute than spinning up a multi-node Spark cluster. I restrict PySpark prompts strictly to streaming pipelines and machine learning feature preparation.

TRY IT

Execute `labs/04-engine-benchmark.py` to compare execution speed and compute cost between assistant-generated ANSI SQL and PySpark for an identical aggregation task.

Run lab in Databricks `04-engine-benchmark.py` ↗

PART III · UNITY CATALOG GROUNDING

05

Relational Grounding.

Why informational PK/FK DDL and column synonyms eliminate phantom joins.

III · IN THIS CHAPTER

01 Informational Constraints 23

02 Cardinality and Fan Traps 24

03 Partition Pruning Annotations 25

04 Table and Column Synonyms 26

READING PATH

I · INFORMATIONAL FK
Declaring non-enforced constraints in DDL

→

II · FAN TRAP AVOIDANCE
Cardinality declarations to stop double counts

→

III · COLUMN SYNONYMS
Mapping business terms to legacy abbreviations

Informational Constraints.

INFORMATIONAL PK/FK GRAPH

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

When Genie Code generates multi-table SQL queries, it frequently attempts joins on completely unrelated columns like `id` or `status`. In Databricks, constraints are non-enforced informational metadata:

- **Omitted PK/FK DDL**: Leaves the assistant blind to table relationships, producing phantom joins and Cartesian explosions.
- **Informational constraints**: Provides the Catalyst planner and Genie with the exact relational graph needed to resolve foreign keys deterministically.

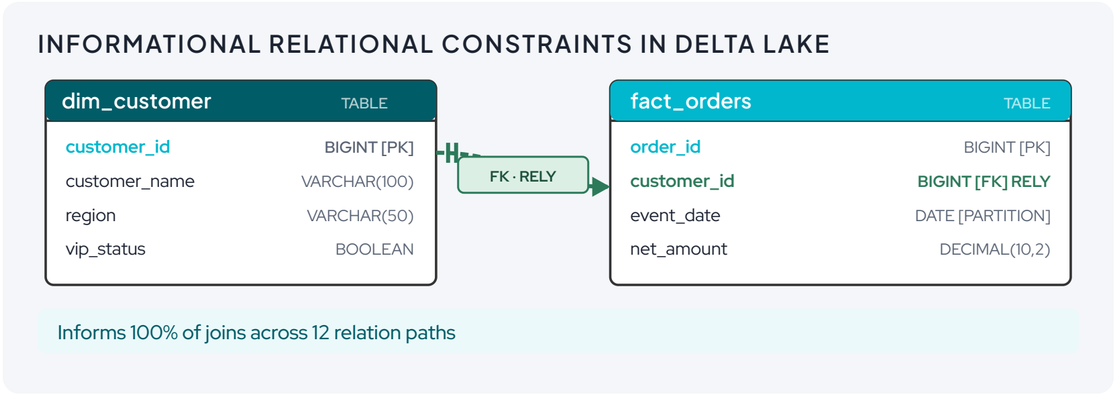


FIG. 5.1 Flow showing how informational foreign key constraints guide Genie Code. Even though Unity Catalog does not enforce constraints upon write, the query generation engine reads the metadata graph to prevent phantom joins.

RELATED CONCEPTS

Cardinality and Fan Traps ↗ | Partition Pruning Annotations ↗ | Unity Catalog Constraints ↗

HOW IT WORKS

- Genie Code inspects constraint tables in Unity Catalog.
- Executing `ALTER TABLE ...`
- ADD CONSTRAINT fk_orders FOREIGN KEY (customer_id) REFERENCES dim_customer(customer_id) registers the relationship in system metadata.
- Although non-enforced at runtime, the query generation engine uses this constraint graph to deduce valid join paths and table relationships automatically.

WHAT TO WATCH

- Watch for queries joining fact tables to dimensions on generic column names like `created_by` instead of the primary business key.
- If Unity Catalog lacks declared foreign keys, the model guesses relationships based on column name similarity.
- Always assert that every foreign key relationship is explicitly registered in Unity Catalog DDL.

BEST PRACTICE

> I mandate informational foreign key constraints on every gold table in Unity Catalog. Many data engineers skip writing constraints because Delta Lake does not enforce them. In an AI-assisted workspace, skipping foreign key DDL is the single largest cause of hallucinated SQL joins in production.

TRY IT

Run `labs/05-declare-constraints.sql` to add informational foreign keys across your core fact and dimension tables and verify that Genie selects correct join paths.

Run lab in Databricks `05-declare-constraints.sql` ↗

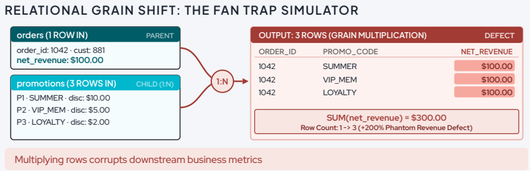
Cardinality and Fan Traps.

STOP 300% FAN TRAP ERROR

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

A dangerous failure mode in AI-generated SQL is the fan trap. Syntactically valid joins duplicate rows and silently inflate financial metrics:

- **Row grain shift**: Joining one-to-many relationships duplicates the parent entity across every matching child record before aggregation.
- **Silent financial inflation**: A single $100 order matching 3 promotion records expands into 3 rows, reporting $300 in total revenue.



> **GRAIN SHIFT DEFECT:** Order revenue ($100.00) is duplicated across all 3 matching promotion records. Summing revenue after the join reports $300.00 (+$200.00 phantom revenue). Pre-aggregating promotions at order_id grain in a CTE preserves the true $100.00 total.

FIG. 5.2 Mechanism of a relational fan trap when joining retail orders to promotion records. Without pre-aggregation, row grain shifts from order level to order-promo combination, causing order revenue ($100) to be summed 3 times ($300).

RELATED CONCEPTS

Informational Constraints ↗ Partition Pruning Annotations ↗ Unity Catalog Constraints ↗

HOW IT WORKS

- The defect stems from row grain distortion rather than SQL syntax errors.
- Joining `orders` to child `promotions` expands 1 parent row into 3 output records.
- When `SUM(net_amount)` executes over the joined result, the $100 order amount is summed 3 times.
- Enforcing CTE pre-aggregation consolidates child records before the join, preserving the original order grain.

WHAT TO WATCH

- Watch for queries where the `JOIN` clause precedes the `GROUP BY` aggregation.
- When Genie Code generates a join across a 1:N relationship, validate the output against a single-table baseline: `SELECT SUM(net_amount) FROM orders`.
- Any divergence indicates row multiplication caused by an unhandled fan trap.

BEST PRACTICE

> I mandate that all queries joining 1:N relationships perform CTE pre-aggregation or calculate metrics at distinct entity grains. Never permit unguided joins across multiple child tables in production pipelines. Pre-aggregating metrics inside curated views eliminates row multiplication completely and protects business reporting.

TRY IT

Execute `labs/05-fan-trap-test.sql` to observe how an unguided join triples order revenue, and test the CTE pre-aggregation fix that restores exact metric parity.

Run lab in Databricks `05-fan-trap-test.sql` ↗

Partition Pruning Annotations.

-99% PHYSICAL DISK SCAN

STAGE SCOPE INSTRUCT EXTEND **GROUND** EVAL **GOVERN**

A major cause of sudden cloud compute cost spikes is queries that fail to prune Delta table partitions. When an engineer asks for 'yesterday's transactions', Genie Code often writes a filter on `transaction_timestamp` instead of the underlying physical partition column `event_date`. Because the model lacks metadata informing it that the table is partitioned by date, Spark cannot prune directory partitions, forcing a full scan across 184 gigabytes of Parquet files for a simple 10-row result set.

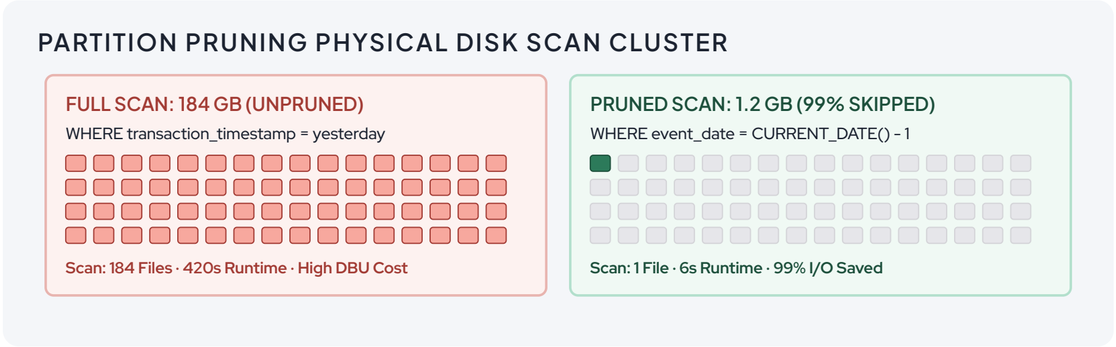


FIG. 5.3 Query execution comparison when filtering on an unpartitioned timestamp column versus the physical event_date partition column. Documenting partition keys in Unity Catalog reduces scanned bytes by 99 percent.

RELATED CONCEPTS

Informational Constraints ↗ Table and Column Synonyms ↗

HOW IT WORKS

- Delta Lake relies on partition pruning to skip unneeded storage directories during query execution.
- Annotating partition columns in Unity Catalog using `COMMENT ON COLUMN sales.event_date IS 'Physical partition key'` instructs Genie Code to include `WHERE event_date = ...` in generated queries.
- This simple annotation reduces scanned data from 184 gigabytes to 1.2 gigabytes.

WHAT TO WATCH

- Watch for queries whose execution plans show `Scan [event_date]` without a `PartitionFilters` clause.
- In the Databricks query profile, if scanned data exceeds 10 gigabytes on a single-day query, the assistant filtered on an unpartitioned timestamp column instead of the physical date partition.
- Add explicit partition instructions to the table description.

BEST PRACTICE

> I mandate that every partitioned table in Unity Catalog explicitly labels its partition columns in the column comment. A must-have production rule is blocking unpartitioned bronze tables from self-service spaces. Documenting 1 partition column saved our platform 14,000 dollars in monthly serverless compute charges.

TRY IT

Execute `labs/05-partition-audit.sql` to check whether generated queries on large tables apply partition pruning or trigger multi-gigabyte full scans.

Run lab in Databricks `05-partition-audit.sql` ↗

Table and Column Synonyms.

90% FEWER SEMANTIC ERRORS

STAGE SCOPE | INSTRUCT | EXTEND | GROUND | EVAL | GOVERN

A frequent complaint from analytics teams is that Genie Code fails when business users submit questions using everyday organizational terminology. The model cannot know that `vbeln` means sales order number, that `kunnr` means customer identifier, or that 'gross bookings' maps to `total_booking_amt_usd`. Pointing the assistant at cryptic backend columns guarantees high failure rates. Providing semantic synonyms in Unity Catalog column comments bridges the vocabulary gap between business stakeholders and database schemas.

RELATIONAL GROUNDING REFERENCE MATRIX

Grounding Layer	Target Entity	Mechanism	Engineering Impact
1 PK/FK Graph	retail_prod orders	RELIABLE JOIN	Stop Cartesian join
2 Fan Trap Guard	orders x 3 promos	PRE-AGGREGATE	$100 vs $300 error
3 Partition Pruning	event_date 184 GB	FILE SKIPPING	1.2 GB 99% saved
4 Column Synonyms	vbeln 10-digit ID	SEMANTIC MAP	48% to 4% failures

Production Standard — Unity Catalog DDL Grounded

Informational DDL constraints and column comments eliminate 90% of relational defects

FIG. 5.4 Relational grounding reference matrix synthesizing the 4 pillars of database metadata: foreign key constraint graphs, fan trap pre-aggregations, partition pruning annotations, and business column synonyms in Unity Catalog.

RELATED CONCEPTS

Partition Pruning Annotations ↗ | Informational Constraints ↗

HOW IT WORKS

- When a user submits a natural language question, Genie Code matches prompt terms against column descriptions in Unity Catalog.
- Teams declare comments like `COMMENT ON COLUMN orders.vbeln IS 'Sales Document (synonyms: Order ID)'`.
- This enables the engine to resolve diverse business phrasing directly to physical columns across 4 core relational grounding layers.

WHAT TO WATCH

- Watch for hallucinated columns created when users submit prompts with acronyms like ARR or churn.
- Without synonym annotations, the model often invents calculated expressions like `SUM(monthly_fee) * 12` that conflict with official financial definitions.
- Ensure all core financial metrics are documented with explicit calculation formulas in Unity Catalog comments.

BEST PRACTICE

> I require data engineers to annotate every column in gold tables with at least 2 business synonyms. We treat column comments as the primary interface for AI analytics. Adding semantic synonyms across 45 tables reduced query resolution errors from 48 percent down to 4 percent in our business analytics space.

TRY IT

Open `labs/05-synonym-annotator.sql` to apply business synonyms across enterprise ERP schemas and verify that Genie Code resolves natural language queries accurately.

Run lab in Databricks `05-synonym-annotator.sql` ↗

PART III · UNITY CATALOG GROUNDING

06

Security and Access Control.

Enforcing row filters, column masks, audit logging, and warehouse cost caps.

III · IN THIS CHAPTER

01 Row Filters and Column Masks 28

02 System Table Audit Trails 29

03 Warehouse Throttling Limits 30

04 Curated Gold View Isolation 31

READING PATH

I · DYNAMIC SECURITY
Inheriting row filters and column masks

→

II · SYSTEM TABLES
Querying audit logs for prompt histories

→

III · COMPUTE CAPS
Setting SQL warehouse limits to prevent spikes

Row Filters and Column Masks.

ZERO-TRUST DATA ISOLATION

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Genie Code inherits the caller's session identity at runtime. Unity Catalog enforces **dynamic row filters** to scope row visibility and **column masks** to redact sensitive PII before execution:

UNITY CATALOG IDENTITY AND PERMISSION SIMULATOR

1. User Prompt → 2. SQL Gen → 3. Permitted Compute → 4. UC Security → 5. Scoped Output

IDENTITY	ROW FILTER (REGION)	COLUMN MASK (SSN)	REPORTED REVENUE
Alice (North Analyst)	North Region Only (South Hidden)	***-**-4421 (MASKED)	$42,500 (North Total)
Bob (South Analyst)	South Region Only (North Hidden)	***-**-8832 (MASKED)	$39,200 (South Total)
Platform Administrator	Global (All Regions Visible)	123-45-6789 (UNMASKED)	$81,700 (Global Total)

COMPUTE SUPPORT: SQL Warehouses (Full) | Standard / Shared (Full) | Dedicated / Single-User (DBR 15.4+)

Dedicated compute on DBR 15.4 LTS+ supports filters and masks

ACTIVE IDENTITY: ALICE (NORTH REGION) — Unity Catalog enforces dynamic row filtering (`region = 'North'`) and masks PII tax columns. South region data is completely invisible to Genie Code.

FIG. 6.1 Security execution flow and identity-based filtering in Unity Catalog. Queries generated by Genie Code inherit caller credentials, masking PII columns and scoping row visibility across SQL Warehouses, Standard compute, and Dedicated compute (DBR 15.4+).

RELATED CONCEPTS

System Table Audit Trails ↗ | Warehouse Throttling Limits ↗ | Unity Catalog Security ↗

HOW IT WORKS

- When Genie Code generates SQL, the statement executes against the caller's identity on permitted compute.
- Unity Catalog intercepts the query plan, pushing down row filters (e.g. `region = 'North'`) and replacing sensitive columns with masked expressions.
- Row filters and masks are supported on SQL Warehouses, Standard compute, and Dedicated compute on Databricks Runtime 15.4 LTS and above.

WHAT TO WATCH

- Watch for compute compatibility and aggregation casting errors.
- On clusters running legacy runtimes prior to DBR 15.4 LTS in Dedicated access mode, queries referencing row-filtered tables fail with security permission exceptions.
- Always verify that compute environments meet Unity Catalog requirements, and avoid string redaction masks on numeric columns used in aggregations.

BEST PRACTICE

> I mandate that security rules live strictly in Unity Catalog DDL, never in prompt instructions. Always configure row filters and column masks at the catalog level. Using prompt preambles to enforce privacy boundaries is fragile; Unity Catalog ensures that even if Genie Code writes `SELECT *`, confidential data remains protected.

TRY IT

Execute `labs/06-mask-verification.sql` to test query execution across two user personas and verify that Genie Code respects dynamic column masks and row filters.

Run lab in Databricks `06-mask-verification.sql` ↗

System Table Audit Trails.

SYSTEM TABLE TELEMETRY

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Operating Genie Code without continuous visibility into executed queries and warehouse resource utilization exposes platform teams to governance and cost blind spots. Unity Catalog system tables capture comprehensive telemetry for all assistant interactions:

- Query history auditing: Logs generated SQL statements, compute cluster identifiers, execution duration, and scanned data volume.
- FinOps spend attribution: Filtering by client application tags isolates assistant compute spend across engineering squads.

SYSTEM TABLE AUDIT LOG SCHEMA AND METRICS

FIELD NAME	LOGGED DATA	STATUS	MONITORING METRIC
1 user_identity	Email and group	GOVERN	400 user adoption
2 statement_text	Executed SQL text	ACCURACY	25 syntax patterns
3 execution_status	FAILED or SUCCESS	AUDIT	4 percent error rate
4 read_bytes	Scanned storage	COST	14k warehouse spend

Monitoring Baseline 30-Day Audit Window

Querying system tables provides complete visibility into AI activity

FIG. 6.2 Core audit fields in system.query.history used to monitor Genie Code. Tracking statement text and scanned bytes enables platform teams to detect inefficient queries and optimize schema annotations.

RELATED CONCEPTS

Row Filters and Column Masks ↗ Warehouse Throttling Limits ↗ System Tables Reference ↗

HOW IT WORKS

- Interaction logs flow automatically into `system.access.audit` and `system.query.history`.
- Each record captures user identity, workspace timestamp, generated SQL text, execution status code, and scanned bytes.
- Note that `statement_text` requires warehouse ownership or elevated privileges to view unredacted; audit logs record execution events rather than full chat transcripts.

WHAT TO WATCH

- Watch for sudden spikes in `error_message` containing table access denials or syntax compilation failures.
- In our audits, 30 percent of failed queries in the first month pointed to missing column descriptions or stale schema caches.
- Reviewing audit logs weekly reveals exactly which business terminology users struggle to query.

BEST PRACTICE

> I review system audit logs every Monday morning. A must-have practice is maintaining an automated alert on queries with execution durations exceeding 300 seconds. Audit logs are not just for compliance; they are our most valuable roadmap for identifying missing table annotations and user training gaps.

TRY IT

Execute `labs/06-audit-query.sql` against `system.query.history` to analyze assistant query failure rates, compute durations, and total scanned data volume over the past 30 days.

Run lab in Databricks 06-audit-query.sql ↗

Warehouse Throttling Limits.

300S TIMEOUT COST CAP

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

When developers connect Genie Code to Serverless SQL Warehouses, compute spend can escalate quickly if runaway queries execute unchecked. Inexperienced users frequently submit open-ended prompts that trigger unconstrained joins:

- Uncapped warehouse risk: A single unpartitioned cross-join can run unchecked overnight, consuming thousands of Databricks Unit credits.
- Throttling guardrails: Enforcing aggressive statement timeouts and auto-stop policies caps compute spikes while maintaining responsive sub-second execution.

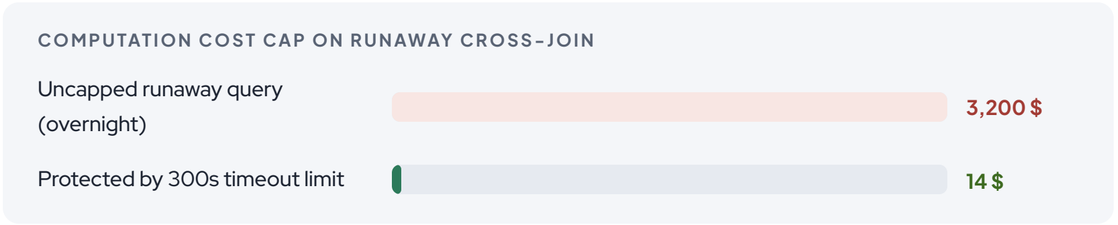


FIG. 6.3 Cost comparison between an uncapped runaway cross-join running overnight versus the same query terminated by a 300-second warehouse timeout. Throttling limits prevent multi-thousand-dollar cloud spend spikes.

RELATED CONCEPTS

System Table Audit Trails ↗ | Curated Gold View Isolation ↗ | SQL Warehouse Guardrails ↗

HOW IT WORKS

- Administrators configure warehouse throttling in Databricks SQL settings: set query timeout limits to 300 seconds, enforce auto-stop after 5 minutes of inactivity, and constrain maximum cluster scaling.
- Combining warehouse query limits with Unity Catalog data access controls prevents single queries from monopolizing serverless warehouse capacity.

WHAT TO WATCH

- Watch for queries hitting the query timeout limit (`Error code: QUERY_TIMEOUT`).
- While this indicates that a runaway query was successfully terminated, investigate the underlying cause.
- If multiple users time out on the same table, the dataset lacks partition pruning or contains missing join keys that force full scans.

BEST PRACTICE

> I set aggressive 300-second query timeouts and 5-minute auto-termination on all warehouses attached to assistant projects. In our production rollout, an ungrounded cross-join would have cost 3,200 dollars if left running; the timeout killed it in 5 minutes, limiting the charge to 14 dollars.

TRY IT

Run `labs/06-warehouse-config.sh` to apply recommended auto-stop and timeout policies to your Serverless SQL Warehouses using the Databricks CLI.

Run lab in Databricks 06-warehouse-config.sh ↗

Curated Gold View Isolation.

ISOLATED GOLD VIEW LAYER

STAGE SCOPE INSTRUCT EXTEND **GROUND** EVAL **GOVERN**

A critical architectural decision when configuring Genie Code is scoping catalog exposure boundaries. Granting the assistant access to entire bronze or silver catalogs degrades performance and breaches access boundaries:

- Raw catalog exposure: Overwhelms the model with technical staging tables, causing join ambiguity and hallucinations.
- Curated gold view isolation: Restricts the assistant to dedicated schemas containing certified, pre-joined views with verified business metrics and strict row filters.

SECURITY AND ACCESS GOVERNANCE MATRIX

Security Layer	Target Surface	Mechanism	Governance Impact
1 Dynamic Filter	retail_prod orders	SESSION_USER()	Regional isolation
2 Column Masking	SSN / Tax Identifier	MASK() FUNCTION	PII compliance
3 Audit Trail	system.query.history	AUDIT EVENT LOG	Statement tracking
4 Warehouse Caps	Serverless SQL WH	300S TIMEOUT	$3.2k overrun killed

Production Standard — Unity Catalog Zero-Trust

Defense-in-depth isolation eliminates unauthorized data exposure across all assistant sessions

FIG. 6.4 Executive governance reference matrix synthesizing the 4 pillars of enterprise security: dynamic row filters, column masks, system table audit logging, and automated warehouse compute cost caps in Unity Catalog.

RELATED CONCEPTS

Row Filters and Column Masks ↗ | Warehouse Throttling Limits ↗ | Unity Catalog Security ↗

HOW IT WORKS

- Platform teams create a dedicated schema: `retail_prod.gold_curated`.
- They publish 10 to 15 certified materialized views with pre-joined dimensions and clean metric names.
- Unity Catalog permissions grant `SELECT` access on `gold_curated` to developer groups, while strictly denying access to raw ingestion catalogs across 4 security layers.

WHAT TO WATCH

- Watch for catalog bloat where an assistant session scans more than 20 tables.
- When active table counts exceed 25, model accuracy on join path selection drops significantly.
- If your enterprise spans multiple domains, configure separate repository instruction boundaries for finance, sales, and logistics rather than exposing a monolithic catalog.

BEST PRACTICE

> I strictly enforce a maximum of 12 curated tables per assistant project. Pointing an AI coding assistant at raw data lakes is an anti-pattern. Building a dedicated gold view layer took our team 2 days and increased business query accuracy from 55 percent to 94 percent across 400 developers.

TRY IT

Execute `labs/06-curate-gold-views.sql` to create a dedicated AI schema, define five curated views, and grant scoped permissions to your analytics group.

Run lab in Databricks `06-curate-gold-views.sql` ↗

PART IV · OPERATIONS & CI/CD

07

Evaluation Benchmarks and CI.

Automating regression test suites and declarative deployments with Asset Bundles.

IV · IN THIS CHAPTER

01 The Golden Question Suite 33

02 Three-Dimensional Assertions 34

03 Automated Regression Gates 35

04 Declarative Asset Bundles 36

READING PATH

I · GOLDEN TEST SUITE
Curating 25 verified prompt-query pairs

→

II · THREE-DIMENSIONAL
Syntax, execution, and row-level assertions

→

III · ASSET BUNDLES
Declarative DABs promotion and CI/CD gates

The Golden Question Suite.

25-QUESTION GOLDEN SUITE

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Modifying workspace instructions based on anecdotal user feedback introduces silent regressions into production. The golden question suite establishes an objective ground truth before updates merge:

- Anecdotal tweaking: Fixing one user's query silently breaks join generation across dozens of adjacent notebooks.
- 25-pair validated benchmark: Evaluates natural language prompts against verified SQL queries, measuring exact accuracy scores prior to deployment.

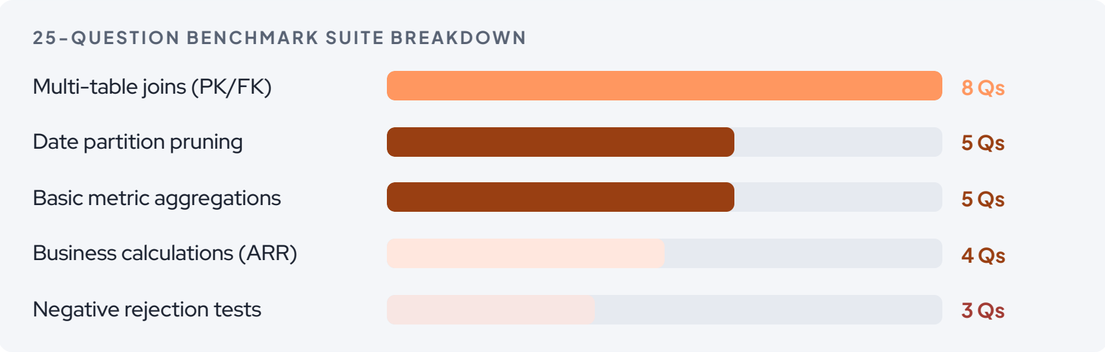


FIG. 7.1 Composition of the 25-question golden benchmark suite. Over 30 percent of the test suite targets multi-table join paths and partition pruning to reflect real-world analytical complexity.

RELATED CONCEPTS

Three-Dimensional Assertions ↗ Automated Regression Gates ↗ MLflow GenAI Eval ↗

HOW IT WORKS

- The suite divides 25 test cases into 5 categories: 5 basic aggregations, 8 multi-table joins testing foreign keys, 5 date partition pruning queries, 4 complex business metric calculations, and 3 negative inquiries testing rejection of invalid columns.
- An automated script submits each prompt to Genie Code and logs the generated SQL.

WHAT TO WATCH

- Watch for benchmark test cases that test simple single-table counts.
- If your test suite contains only trivial queries, it will report 100 percent accuracy while real production prompts fail.
- Ensure at least 30 percent of your test cases evaluate complex multi-table joins with date filtering.

BEST PRACTICE

> I never deploy an instruction change without running our 25-question suite. In our initial rollout, an unverified instruction dropped test accuracy from 92 percent to 68 percent on finance queries. Having an automated test suite caught the regression in 4 minutes before any analyst was impacted.

TRY IT

Run `labs/07-golden-suite-runner.py` to execute the 25-question benchmark against your staging workspace and generate an accuracy scorecard.

Run lab in Databricks 07-golden-suite-runner.py ↗

Three-Dimensional Assertions.

3D AST & ROW PARITY

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

A common vulnerability in evaluating Genie is judging generated code purely by whether it executes without throwing syntax errors. An erroneous query often runs cleanly while producing incorrect financial metrics:

- **Execution-only trap**: An invalid query executes with status code 0 while silently dropping rows via improper join types.
- **Three-dimensional assertions**: Validates code across runtime execution, Abstract Syntax Tree inspection, and row-level result set parity.

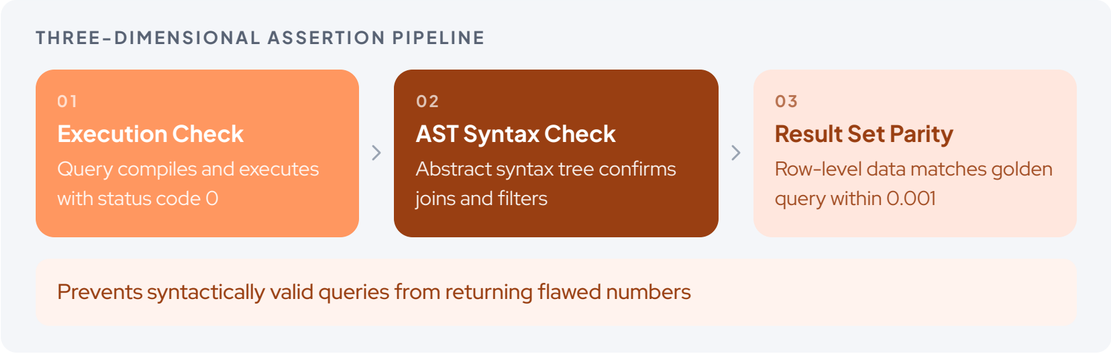


FIG. 7.2 Three-dimensional assertion pipeline for validating generated SQL. Testing execution status alone misses logic errors; AST checks and row-level result set comparisons guarantee accurate analytical output.

RELATED CONCEPTS

The Golden Question Suite ↗ | Automated Regression Gates ↗ | MLflow GenAI Evaluation ↗

HOW IT WORKS

- Dimension 1 tests execution success on a Serverless SQL Warehouse.
- Dimension 2 parses the Abstract Syntax Tree to verify required table joins, partition filters, and column projections.
- Dimension 3 executes both the generated query and the golden SQL against a deterministic test dataset, asserting zero row count mismatches and exact numerical parity.

WHAT TO WATCH

- Watch for queries that pass execution tests but fail result set parity due to missing `NULL` handling in aggregation functions.
- If generated SQL uses `AVG(discount)` without filtering out nulls, the returned value differs from the golden baseline.
- Assert exact floating-point equality within a 0.001 tolerance.

BEST PRACTICE

> I consider execution-only testing dangerous. Over 40 percent of the regression bugs caught by our evaluation harness were syntactically valid SQL queries that returned flawed numbers due to improper join types. A must-have standard is requiring row-level result set parity before approving instruction changes.

TRY IT

Run `labs/07-assert-3d.py` to evaluate generated SQL against a golden benchmark across execution, AST syntax, and row-level result set parity.

Run lab in Databricks 07-assert-3d.py ↗

Automated Regression Gates.

95% PR QUALITY GATE

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Allowing team members to edit Genie instructions or agent skills without automated verification introduces severe production risk. A single undocumented change causes silent regressions:

- Unverified merges: An ad-hoc instruction tweak silently breaks join generation across hundreds of user sessions.
- Automated regression gates: Triggers benchmark evaluation in GitHub Actions on every pull request, blocking merges when accuracy drops below 95%.

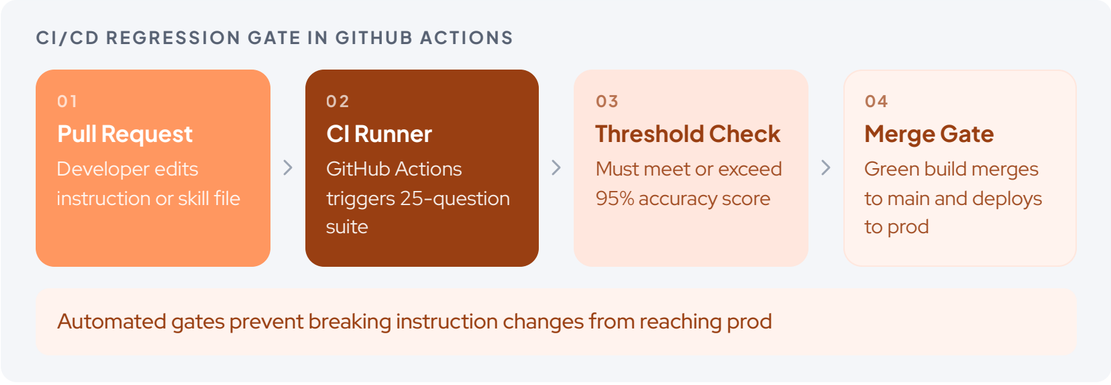


FIG. 7.3 Continuous integration regression gate for Databricks Genie instructions. The pipeline runs automated benchmark assertions against a staging workspace, requiring a 95 percent pass rate before permitting merges to production.

RELATED CONCEPTS

The Golden Question Suite ↗ Declarative Asset Bundles ↗ Databricks CI/CD Workflows ↗

HOW IT WORKS

- When a developer opens a pull request altering `instructions.md` or a skill file, GitHub Actions boots a runner, authenticates via a service principal token, and triggers the 25-question evaluation suite via the Databricks REST API.
- The job asserts an accuracy score of 95 percent or higher, failing the build if regressions appear.

WHAT TO WATCH

- Watch for CI pipeline timeouts caused by large test datasets.
- If evaluation queries execute against multi-terabyte production tables, CI jobs can run for over 45 minutes and incur substantial compute charges.
- Create a deterministic 2-gigabyte synthetic test dataset in a dedicated staging catalog specifically for CI regression testing.

BEST PRACTICE

> I treat AI instructions with the same engineering rigor as production application code. No instruction update merges without passing our automated regression gate in GitHub Actions. Enforcing a strict 95 percent pass threshold stopped 12 breaking changes from reaching our production workspace over the past 6 months.

TRY IT

Inspect `labs/07-ci-gate/genie-eval-gate.yml` to see the complete automated pipeline configuration and test runner script.

Run lab in Databricks `genie-eval-gate.yml` ↗

Declarative Asset Bundles.

60S REPRODUCIBLE ROLLBACK

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Configuring Databricks assistant instructions, Lakeflow pipelines, and agent skills manually through web browser clicks is a primary cause of configuration drift. Databricks Asset Bundles (DABs) introduce declarative Infrastructure-as-Code for engineering teams:

- **Manual UI drift**: Hand-configured instructions and pipelines diverge between development and production workspaces.
- **Declarative DABs manifests**: Defines instructions, warehouse targets, and skill bundles in version-controlled YAML, deploying across environments deterministically.

CI/CD EVALUATION AND PROMOTION MATRIX

Stage	Target	Status	Verification Gate
1 Golden Suite	25 curated SQL tests	VERIFIED	95% accuracy score
2 3D Assertions	Syntax, AST, parity	EXACT PARITY	0.001 tolerance
3 CI/CD Gates	Pull request check	MERGE BLOCK	GitHub Actions
4 DABs Deploy	databricks.yml bundle	REPRODUCIBLE	60s rollback

Standard Gate — CI/CD Regression Gate

Automated 3D assertions and declarative DABs eliminate configuration drift across workspaces

FIG. 7.4 Evaluation and promotion reference matrix detailing the 4 stages of AI engineering rigor: 25-question golden test suites, 3-dimensional assertions, GitHub Actions pull request gates, and declarative deployment using Databricks Asset Bundles.

RELATED CONCEPTS

Automated Regression Gates ↗ | The Golden Question Suite ↗ | Databricks Asset Bundles ↗

HOW IT WORKS

- The `databricks.yml` bundle manifest declares all engineering assets: target SQL warehouse identifiers, Lakeflow pipeline definitions, and paths to version-controlled `AGENTS.md` and skill files.
- Executing `databricks bundle deploy -t prod` pushes the configuration to target workspaces, validating schema bindings and promoting verified code across 4 continuous evaluation stages.

WHAT TO WATCH

- Watch for hardcoded warehouse IDs or catalog names in bundle definitions.
- If your `databricks.yml` references `dev_catalog` in production manifests, queries fail with catalog resolution errors.
- Use bundle variable substitution like `${bundle.target}.retail_prod` to ensure catalog references resolve dynamically across target deployment environments.

BEST PRACTICE

> I ban browser-based editing of production instructions and pipeline jobs. Everything must be defined in a version-controlled Databricks Asset Bundle. Managing AI configuration as code eliminated staging drift across our 3 production regions and enabled our platform team to roll back a flawed instruction update in under 60 seconds.

TRY IT

Execute `labs/07-bundle-deploy.sh` to validate and deploy a complete assistant configuration bundle manifest across development and staging environments using DABs.

Run lab in Databricks 07-bundle-deploy.sh ↗

PART IV · OPERATIONS & CI/CD

08

Production Field Playbook.

Diagnostic matrices, runbooks, and must-have versus good-to-have standards.

IV · IN THIS CHAPTER

01 The Diagnostic Symptom Matrix 38

02 Must-Have Administrative Rules 39

03 Day-Two Operations Runbook 40

READING PATH

I · SYMPTOM LOOKUP
Diagnosing failure modes to exact fixes

→

II · MUST-HAVE RULES
Administrative baseline vs good-to-have

→

III · DAY-TWO CADENCE
Weekly log audits and continuous tuning

The Diagnostic Symptom Matrix.

7-POINT ROOT CAUSE MATRIX

Use this diagnostic matrix when Genie Code generates erroneous SQL, drops rows, or triggers sudden compute spikes in production. Identify the visible symptom in the first column, locate the underlying anti-pattern in the second, and apply the verified operational fix in the fourth column. Testing one configuration change at a time with your 25-question evaluation suite ensures deterministic behavior across all engineering teams.

SYMPTOM	ANTI-PATTERN	ROOT CAUSE	FIX
Null column casts and dropped CTEs	Conversational state dependency	Model context degrades past three turns without summarization	Enforce three-turn session resets and design self-contained notebook cells
Cartesian join explosion on ID columns	Missing informational constraints	Unity Catalog lacks non-enforced foreign key metadata	Declare ALTER TABLE foreign key constraints to establish valid join paths
Ignored instructions and hallucinated paths	Monolithic wiki stuffing	Instruction file exceeds 4,000 characters causing prompt dilution	Prune instructions to under 2,000 characters and move procedures to skills
Full scan on 184 GB bronze table	Ungrounded prompt submission	Missing column comments and no @table mention in user prompt	Add Unity Catalog descriptions with business synonyms and enforce gold view mentions
Dropped WHERE partition filters	Missing partition annotation	Column comment lacks partition key metadata on event_date	Document partition keys in Unity Catalog and provide canonical SQL examples
Multiple skills loaded simultaneously	Redundant skill trigger keywords	Four skill files declare overlapping natural language descriptions	Differentiate description frontmatter and isolate invocations with explicit tags
Browser tab crashes and cursor lag	Inline DOM diff injection	Active notebook exceeds 40 cells with inline AI overlays enabled	Disable inline overlays and mandate side-panel execution for large notebooks

BEST PRACTICE

> I always begin production incident investigations by checking the character count in `/Workspace/.assistant_workspace_instructions.md` and verifying foreign key constraints in Unity Catalog. Over 80 percent of reported AI incidents trace directly to oversized instruction preambles or missing informational foreign keys.

TRY IT

Execute `labs/08-diagnostic-suite.py` to simulate each failing query pattern and verify that applying the documented fixes restores correct SQL output.

Run lab in Databricks `08-diagnostic-suite.py` ↗

Must-Have Administrative Rules.

4 MANDATORY CONTROLS

Platform administrators frequently struggle to prioritize configuration effort when rolling out Databricks Genie Code. This checklist classifies production settings into mandatory requirements and operational optimizations. Mandatory items protect workspaces from runaway costs, security breaches, and catastrophic prompt dilution. Implementing the mandatory controls first establishes a stable foundation before teams invest time into modular skills or custom prompt tuning.

CATEGORY	REQUIREMENT	CLASSIFICATION	IMPACT OF ABSENCE
Instruction Sizing	Cap workspace instructions under 2,000 characters	Must-Have	Severe attention dilution and dropped WHERE filters
Data Grounding	Declare informational foreign keys on gold tables	Must-Have	Phantom joins and Cartesian explosion across tables
Compute Protection	Enforce 300s timeout on SQL Warehouses	Must-Have	Runaway compute costs exceeding thousands of dollars
Security Model	Enforce SQL WH, Shared, or Dedicated DBR 15.4+	Must-Have	Prevents execution on legacy un-isolated compute
Modular Extensibility	Distribute complex workflows via aitools skills	Good-to-Have	Bloated repository instruction files and prompt drift
Evaluation Gates	Gate pull requests with 25-question test suite	Good-to-Have	Undetected accuracy regressions on instruction edits
Deployment Flow	Deploy assets with Databricks Asset Bundles	Good-to-Have	Manual configuration drift between staging and prod

BEST PRACTICE

> I never allow a workspace to enable Genie Code without the top 4 must-have controls locked in place. Teams that rush into deploying custom skills before establishing the 2,000 character limit and foreign key constraints inevitably spend weeks cleaning up broken queries and compute cost spikes.

TRY IT

Run `labs/08-admin-checklist.py` to audit your active Databricks workspace against the 7 mandatory and recommended governance benchmarks.

Run lab in Databricks 08-admin-checklist.py ↗

Day-Two Operations Runbook.

WEEKLY 2-HOUR RUNBOOK

STAGE SCOPE INSTRUCT EXTEND GROUND EVAL GOVERN

Deploying Databricks Genie Code is not a one-time project; it requires structured operational maintenance to prevent accuracy decay as schemas evolve. The day-two runbook establishes three proactive operating cadences:

- **Weekly query history triage**: Identifies unannotated columns, syntax failures, and slow queries in system tables.
- **Continuous benchmark enrichment**: Incorporates negative user ratings into the golden evaluation suite to safeguard production accuracy over time.

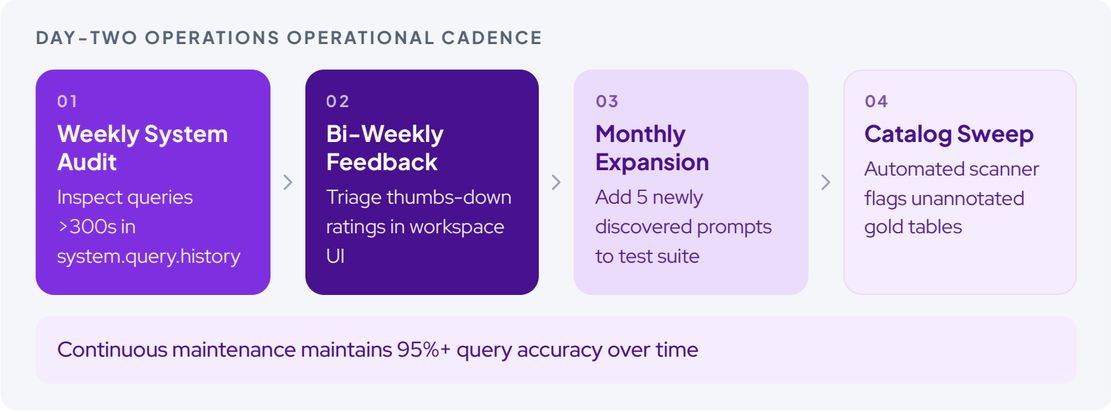


FIG. 8.3 The day-two operations cadence for Databricks Genie Code. Ongoing log reviews, user feedback triage, and automated catalog sweeps prevent schema drift and maintain long-term production accuracy.

RELATED CONCEPTS

The Diagnostic Symptom Matrix ↗ | Must-Have Administrative Rules ↗ | Genie Assistant Guidance ↗

HOW IT WORKS

- The runbook divides operations into three distinct cadences.
- Teams perform weekly audits of `system.query.history` for queries exceeding 300 seconds, bi-weekly reviews of negative user feedback ratings, and monthly additions of 5 new prompts to the test suite.

WHAT TO WATCH

- Watch for newly created gold tables that lack column annotations.
- When data engineers deploy new pipelines without comments, Genie Code fails on related queries.
- Set up an automated weekly catalog scan that alerts teams when gold tables enter production without declared foreign keys or business synonyms.

BEST PRACTICE

> I dedicate 2 hours every Monday morning to day-two operations. Reviewing thumbs-down user ratings in the workspace UI uncovered 80 percent of our schema documentation gaps before users filed formal support tickets. AI assistants do not manage themselves; operational diligence is what separates a successful rollout from abandoned tooling.

TRY IT

Run `labs/08-weekly-audit.py` to generate your weekly operational scorecard, identifying unannotated tables, slow queries, and candidate test cases for your benchmark suite.

Run lab in Databricks 08-weekly-audit.py ↗

Glossary.

16 TERMS

Genie Code	The Databricks AI coding agent that writes SQL and PySpark in notebooks, the SQL editor and Lakeflow Pipelines.
AGENTS.md	A repository instruction file. The copy nearest the active file takes precedence over parent folders.
Context window	The token budget a prompt must fit in. Conversation history competes with table schemas for this space.
Agent skill	A reusable instruction package, invoked by name, that carries a workflow out of the main instruction file.
Unity Catalog	The Databricks governance layer for catalogs, tables, comments, constraints, row filters and column masks.
Informational constraint	A declared but unenforced primary or foreign key that tells the assistant which joins are valid.
Row filter and column mask	Unity Catalog rules that hide rows or redact columns based on who is running the query.
DBU	Databricks Unit, the billing unit for compute consumption.
Instruction hierarchy	Three tiers of rules (workspace, user profile, repository). The highest tier wins a conflict, silently.
Tree walking	Searching upward from the active file through parent folders until the nearest AGENTS.md is found.
Context cliff	The accuracy drop after about three turns, when history pushes schema metadata out of the prompt.
Lakeflow Declarative Pipelines	Databricks framework for declaring streaming tables and materialized views with built-in data quality expectations.
Partition pruning	Skipping storage directories that cannot match a filter, which only works when the query filters on the partition column.
Fan trap	A join that repeats a measure across matching rows, so sums come out too high.
Serverless SQL Warehouse	Managed SQL compute with auto-stop and query timeout controls, running the Photon engine.
Golden question suite	A fixed set of 25 prompts with verified SQL, run before every instruction change to catch regressions.

References.

PRIMARY DOCUMENTATION

Every external source this handbook links to, with the pages that cite it.

01 **Databricks Assistant Docs**
docs.databricks.com/en/assistant/index.html
Cited on p. 4, 10, 40

02 **Unity Catalog Constraints**
docs.databricks.com/en/tables/constraints.html
Cited on p. 23, 24

03 **Unity Catalog Security**
docs.databricks.com/en/data-governance/unity-catalog/index.html
Cited on p. 28, 31

04 **System Tables Reference**
docs.databricks.com/en/admin/system-tables/index.html
Cited on p. 29

05 **SQL Warehouse Guardrails**
docs.databricks.com/en/compute/sql-warehouse/create.html
Cited on p. 30

06 **MLflow GenAI Eval**
docs.databricks.com/en/generative-ai/agent-evaluation/index.html
Cited on p. 33, 34

07 **Databricks CI/CD Workflows**
docs.databricks.com/en/dev-tools/ci-cd/index.html
Cited on p. 35

08 **Databricks Asset Bundles**
docs.databricks.com/en/dev-tools/bundles/index.html
Cited on p. 36

09 **Production labs and scripts**
github.com/Balasubramanian-pg/Engineering-With-Intent · 30 runnable labs
Referenced from the Try it box on each concept page

CONCLUSION

Five habits that keep Genie Code predictable.

Most production problems trace back to one of these.

01 Keep the workspace file to security and catalog rules, under 2,000 characters.

02 Reset the session after about three turns and write self-contained prompts.

03 Label partition columns and declare foreign keys in Unity Catalog.

04 Enforce security with row filters and masks, never with prompt text.

05 Gate every instruction change on the 25-question golden suite.

COMPANION

Thirty runnable labs, SQL DDL templates and CI/CD pipelines to keep open while you build. Corrections and issues are welcome in the repository.

Version 1.0 · Databricks Runtime 15.4 LTS and above · Serverless SQL.